



L OVELY
P ROFESSIONAL
U NIVERSITY

Name: Anmol Singh
Registration No: 12323397
Program: B.Tech CSE
University: Lovely Professional University,
Phagwara, Punjab
Section: 20M44
Roll No.: 04
Course Code: INT377
Course Title: DEVOPS AND CLOUD
Faculty Guide: Ajay Kumar

Link: [GitHub](#)

1. Introduction

Modern software development requires automation, security, scalability, and continuous delivery to deploy applications efficiently. Traditional deployment methods are slow, manual, and prone to errors. They also lack security validation, monitoring, and automated rollback mechanisms.

This project presents a complete **Production-Grade DevSecOps CI/CD Pipeline** integrated with **GitOps principles** using Jenkins and ArgoCD. The system automates the complete software delivery lifecycle including:

- Source code integration
- Automated testing
- Security scanning
- Artifact management
- Docker containerization
- Kubernetes deployment
- Monitoring and alerting
- Notification management

The project demonstrates how modern DevOps and cloud-native tools work together to achieve reliable, secure, and automated software deployment in a production-like environment.

2. Problem Statement

Traditional software deployment systems face several limitations:

- Manual deployment increases operational effort
- Lack of automated testing causes unstable releases
- Security vulnerabilities remain undetected
- No centralized monitoring and observability

- No GitOps-based deployment strategy
- Difficult to manage scalable cloud-native deployments

Therefore, organizations require an automated DevSecOps ecosystem that can:

- Automate CI/CD workflows
- Perform security scanning
- Support Kubernetes deployments
- Implement GitOps methodology
- Provide real-time monitoring and notifications

3. Objectives

The main objectives of this project are:

- To build a production-grade DevSecOps CI/CD pipeline
- To automate build, test, scan, and deployment processes
- To integrate Jenkins for Continuous Integration
- To implement GitOps deployment using ArgoCD
- To perform code quality analysis using SonarQube
- To scan Docker images using Trivy
- To store artifacts in JFrog Artifactory
- To deploy applications on AWS EKS Cluster
- To monitor infrastructure using Prometheus and Grafana
- To integrate Slack notifications for alerts
- To securely manage credentials using Hashicorp Vault

4. Tools & Technologies Used

Version Control

- Git

- GitHub

CI/CD Tools

- Jenkins
- ArgoCD

Build & Testing

- Maven
- JUnit

DevSecOps & Security

- SonarQube
- Trivy
- Hashicorp Vault

Containerization & Orchestration

- Docker
- Kubernetes (EKS)
- Helm

Artifact Management

- JFrog Artifactory

Cloud Services

- AWS EC2
- AWS EKS
- AWS S3
- Terraform

Monitoring & Notifications

- Prometheus
- Grafana
- Slack

5. System Architecture

Architecture Overview

This project follows a GitOps-based DevSecOps architecture where Jenkins handles Continuous Integration and ArgoCD handles Continuous Deployment.

Complete Pipeline Flow

1. Developer pushes code to GitHub repository
2. GitHub webhook triggers Jenkins pipeline
3. Jenkins performs Maven build
4. JUnit executes unit test cases
5. SonarQube performs code quality analysis
6. Quality Gate validates code standards
7. Artifacts are uploaded to JFrog Artifactory
8. Docker image is built
9. Trivy scans Docker image for vulnerabilities
10. Scan reports are uploaded to AWS S3
11. Docker image is pushed to Docker Hub
12. Jenkins updates Kubernetes manifest files
13. Jenkins creates Pull Request in manifest repository
14. Team reviewer approves and merges Pull Request
15. ArgoCD detects Git changes automatically
16. ArgoCD deploys application into EKS Cluster
17. Prometheus and Grafana monitor infrastructure
18. Slack sends build and deployment notifications

6. Main Workflow of the Project

Step 1: Source Code Management

The application source code is stored in GitHub repositories. Developers push code changes into the repository.

Step 2: Continuous Integration using Jenkins

GitHub webhook automatically triggers Jenkins whenever code changes occur.

Jenkins performs:

- Code checkout
- Maven build
- JUnit testing
- SonarQube analysis
- Quality Gate validation

If any stage fails, the pipeline stops and Slack sends failure notifications.

Step 3: Artifact Management

After successful validation:

- Build artifacts (.jar files) are uploaded to JFrog Artifactory
- Artifacts are securely stored for future deployments

Step 4: Dockerization & Security Scanning

Jenkins builds Docker images from application source code.

Trivy scans Docker images for:

- Vulnerabilities
- Security risks
- Misconfigurations

Generated reports are uploaded to AWS S3 for auditing purposes.

Step 5: Docker Image Deployment

Validated Docker images are pushed into Docker Hub repository with version tags.

Step 6: GitOps Deployment

Jenkins updates Kubernetes deployment manifests automatically.

Instead of directly deploying changes:

- Jenkins creates a Pull Request
- Senior team members review the changes
- After approval, PR is merged into the main branch

This ensures controlled production deployments.

Step 7: Continuous Deployment using ArgoCD

ArgoCD continuously monitors the Kubernetes manifest repository.

Whenever changes are merged into the main branch:

- ArgoCD detects the updates
- Pulls latest manifests
- Deploys applications into EKS cluster automatically

Step 8: Monitoring & Notifications

Prometheus and Grafana continuously monitor:

- Cluster health
- CPU usage
- Memory usage
- Pod status
- Application availability

Slack provides:

- Build success notifications
- Deployment alerts
- Failure alerts

7. Server Configuration Used

Server	Purpose	Instance Type
Jenkins Server	Jenkins, Docker, Trivy, Terraform	t2.large
SonarQube Server	SonarQube & Vault	t2.medium
Artifactory Server	JFrog Artifactory	t2.medium
EKS Cluster	Kubernetes Deployment	t3.medium Nodes

8. Pipeline Stages

CI/CD PIPELINE – 15 STAGES

STAGE NO.	STAGE NAME	DESCRIPTION	TOOLS / TECHNOLOGIES USED
01	 GIT CHECKOUT	Source code is cloned from GitHub repository.	 GitHub
02	 BUILD & UNIT TESTING	Maven builds the application and JUnit executes unit test cases.	 
03	 SONARQUBE ANALYSIS	SonarQube analyzes the code quality, bugs, vulnerabilities and code smells.	 SonarQube
04	 QUALITY GATE VALIDATION	Pipeline waits for SonarQube Quality Gate result. If failed, pipeline stops.	 SonarQube Quality Gate 
05	 ARTIFACT UPLOAD TO ARTIFACTORY	Build artifacts (.jar) are uploaded to JFrog Artifactory repository.	 JFrog Artifactory
06	 DOCKER BUILD	Docker image is built from the application source code using Dockerfile.	 Docker
07	 TRIVY SECURITY SCAN	Trivy scans the Docker image for vulnerabilities and security issues.	 Trivy
08	 S3 REPORT UPLOAD	Trivy scan report is uploaded to AWS S3 bucket for future reference.	 AWS S3
09	 DOCKER HUB PUSH	Docker image is pushed to Docker Hub repository.	 Docker Hub
10	 CLONE / PULL MANIFEST REPO	Jenkins clones or pulls the Kubernetes manifest repository from feature branch.	 GitHub
11	 UPDATE MANIFEST	Image tag in the deployment manifest (deployment.yaml) is updated.	 Kubernetes Manifest
12	 COMMIT & PUSH CHANGES	Updated manifest is committed and pushed to the feature branch.	 GitHub
13	 CREATE PULL REQUEST	Jenkins creates a Pull Request against main branch.	 GitHub Pull Request 
14	 ARGOCD DEPLOYMENT	ArgoCD detects changes in main branch and deploys application to EKS Cluster.	 argo  Amazon EKS
15	 SLACK NOTIFICATION	Slack sends success/failure notification about build and deployment status.	 Slack



SUMMARY: This 15 stage pipeline automates the complete CI/CD process with DevSecOps and GitOps practices to deliver secure, reliable and production-ready applications.

9. Security Implementation

This project follows DevSecOps best practices:

- SonarQube for code quality and vulnerability scanning
- Trivy for Docker image security scanning
- Hashicorp Vault for secret management
- GitOps workflow for controlled deployments
- Pull Request approval mechanism before production deployment

10. Monitoring & Observability

Prometheus

Prometheus collects metrics from:

- Kubernetes cluster
- Nodes
- Pods
- Containers

Grafana

Grafana visualizes monitoring metrics through dashboards.

Monitored parameters include:

- CPU utilization
- Memory usage
- Pod health
- Network traffic
- Application uptime

11. Expected Outcomes

- Fully automated CI/CD pipeline
- Secure production-like DevSecOps workflow
- Automated testing and deployment
- Secure Docker image deployment
- Real-time monitoring and alerts
- Reduced manual intervention
- Faster software delivery lifecycle
- Improved deployment reliability

12. Applications

This project can be used in:

- Enterprise CI/CD automation
- Kubernetes production environments
- Cloud-native application deployments
- DevSecOps implementation
- GitOps workflows
- Infrastructure monitoring systems

13. Future Scope

Future improvements can include:

- Multi-cloud deployment support
- AI-based anomaly detection
- Automated rollback mechanisms
- ELK Stack centralized logging
- Blue-Green deployments
- Canary deployments
- Service Mesh using Istio

14. Main Flow Diagram

Simple Project Flow



15. Screenshots Section

- Jenkins CI pipeline configuration with GitHub integration

The screenshot shows a GitHub repository page for 'DevOps_Mastery-CI-with-Jenkins' by user 'Anmol-Singh-0016'. The repository is public and has 15 commits. The commit history table lists recent updates to files like 'images', 'src', 'Dockerfile', 'Jenkinsfile', 'README.md', 'mvnw', 'mvnw.cmd', and 'pom.xml'. The README.md file is open, showing the title 'DevOps-Mastery' and a brief description. The right sidebar contains an 'About' section describing the project as a DevOps-Mastery Project using various tools like Git, GitHub, Jenkins, Maven, JUnit, SonarQube, Jfrog Artifactory, Docker, Trivy, AWS S3, Docker Hub, GitHub CLI, EKS, ArgoCD, Prometheus, Grafana, Slack, and Hashicorp Vault. It also shows social media links for 'anmolsingh.dev' and statistics like 0 stars, 1 watching, and 0 forks.

The screenshot shows the Jenkins CI integration configuration page for Slack. The page title is 'Jenkins CI' and it describes Jenkins as an open source continuous integration server. It states that Jenkins CI is a customizable continuous integration server with over 600 plugins, allowing you to configure it to meet your needs. This integration will post build notifications to a channel in Slack. The 'Post to Channel' section prompts the user to start by choosing a channel where Jenkins notifications will be posted. A dropdown menu labeled 'Choose a channel...' is shown. Below the dropdown, there is a link 'or create a new channel'. At the bottom, there is a button labeled 'Add Jenkins CI integration'. The footer contains links for 'USING SLACK', 'SLACK', 'LEGAL', and 'HANDY LINKS'.

- **SonarQube main dashboard with quality gates and project analysis**

SonarQube

[Projects](#)
[Issues](#)
[Rules](#)
[Quality Profiles](#)
[Quality Gates](#)
[Administration](#)
[More](#)

Filters

My Favorites All

Quality Gate

Passed	1
Failed	0

Reliability (🐛 Bugs)

A rating	0
B rating	0
C rating	1
D rating	0
E rating	0

Security (🔒 Vulnerabilities)

A rating	0
B rating	0
C rating	0
D rating	1
E rating	0

Security Review (🛡️ Security Hotspots)

A > 80%	1
B 70% - 80%	0
C 50% - 70%	0
D 30% - 50%	0
E < 30%	0

Maintainability (⚙️ Code Smells)

A rating	1
B rating	0

[Create Project](#)

1 project(s)

Perspective: Overall Status ▾

Sort by: Name ▾

gitops-with-argocd
Passed

Last analysis: 9 hours ago

Bugs
 1 C

Vulnerabilities
 1 D

Hotspots Reviewed
 — A

Code Smells
 6 A

Coverage
0.0% F

Duplications
0.0% A

Lines
194 XS Java, XML

1 of 1 shown

Embedded database should be used for evaluation purposes only

The embedded database will not scale. It will not support upgrading to newer versions of SonarQube, and there is no support for migrating your data out of it into a different database engine.

SonarQube™ technology is powered by [SonarSource SA](#)

Community Edition - Version 10.1 (build 73491) - [LGPL v3](#) - [Community](#) - [Documentation](#) - [🔗 Rims](#) - [Plugins](#) - [Web API](#)

- **JFrog Artifactory project creation and webhook setup**

sonarqube

ProjectsIssuesRulesQuality ProfilesQuality GatesAdministration

Search for projects...

?

🔔

A

Create a project

All fields marked with * are required

Project display name *

Up to 255 characters. Some scanners might override the value you provide.

Project key *

The project key is a unique identifier for your project. It may contain up to 400 characters. Allowed characters are alphanumeric, '-' (dash), '_' (underscore), '.' (period) and ':' (colon), with at least one non-digit.

Set Up

sonarqube

demo master

February 26, 2023 at 12:52 AM Version 0.9.1-SNAPSHOT

OverviewIssuesSecurity HotspotsMeasuresCodeActivity

Webhooks

Webhooks are used to notify external services when a project analysis is to each of the provided URLs. Learn more in the Webhooks documentation

Name	URL
test	http://localhost:8081/sonarqube-webhook/

Create Webhook

All fields marked with * are required

Name *

test

✓

URL *

http://localhost:8081/sonarqube-webhook/

✓

The webhook payload is a JSON object. For more details on the webhook payload structure, consult the documentation.

Example: http://localhost:8081/sonarqube-webhook/

Secret

✓

If provided, secret will be used as the key to generate the HMAC hex (lowercase) digest value in the 'X-Sonar-Webhook-HMAC-SHA256' header

CreateCancel

- **DockerHub repository with tags and push commands**

Anmol-Singh-0016 / spring-app

General

Tags

Builds

Collaborators

Webhooks

Settings

Add a short description for this repository

[Update](#)

The short description is used to index your content on Docker Hub and in search engines. It's visible to users in search results.

Anmol-Singh-0016 / spring-app

Description

This repository does not have a description.

Last pushed 2 hours ago

Docker commands

Public View

To push a new tag to this repository,

```
docker push Anmol-Singh-0016/spring-app:tagname
```

Tags

This repository contains 25+ tag(s).

Tag	OS	Type	Pulled	Pushed
46c9d7016d8e00856a...		Image	—	2 hours ago
45c9d7016d8e00856a...		Image	2 hours ago	5 hours ago
44c9d7016d8e00856a...		Image	4 hours ago	5 hours ago
43c9d7016d8e00856a...		Image	4 hours ago	5 hours ago
42c9d7016d8e00856a...		Image	4 hours ago	5 hours ago

[See all](#)[Go to Advanced Image Management](#)

Automated Builds

Manually pushing images to Hub? Connect your account to GitHub or Bitbucket to automatically build and tag new images whenever your code is updated, so you can focus your time on creating.

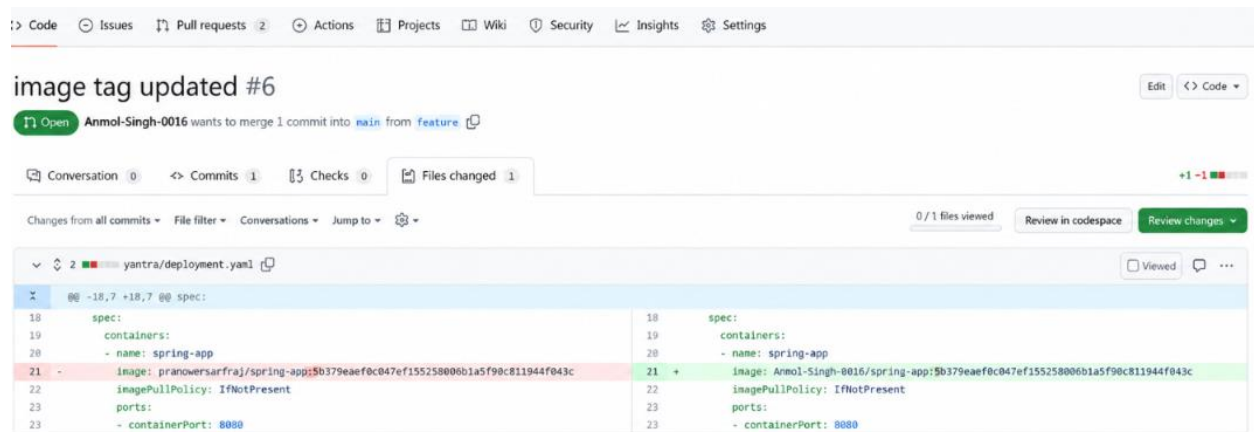
Available with Pro, Team and Business subscriptions. [Read more about automated builds](#)

Upgrade

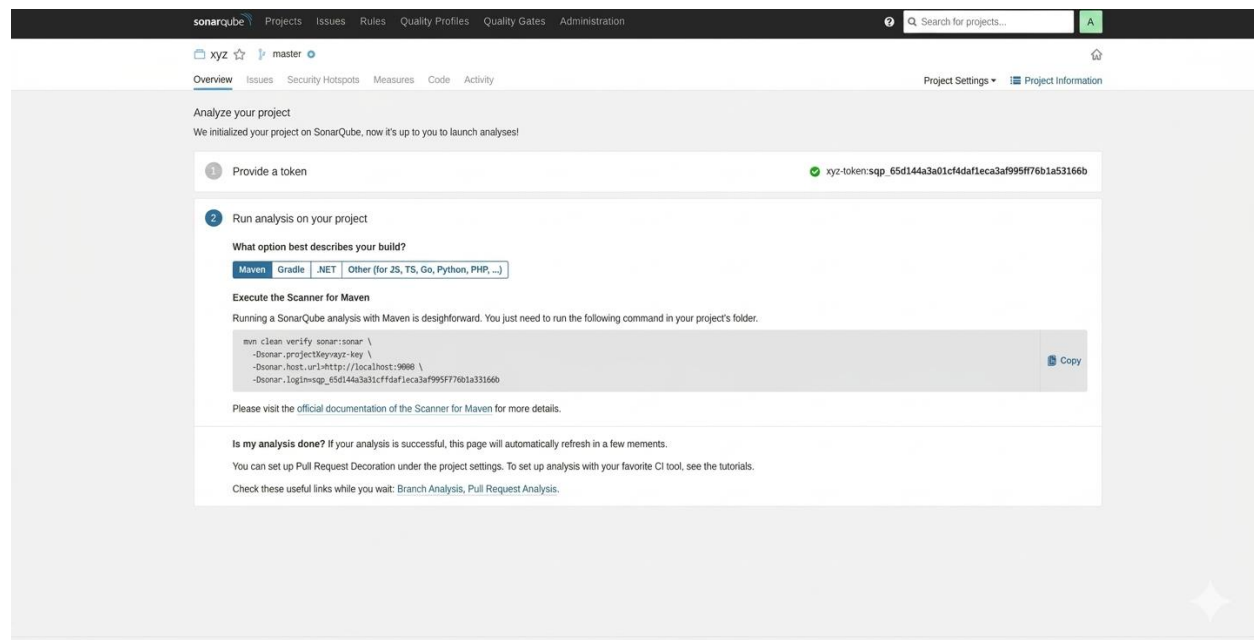
- **Trivy vulnerability scan report with CVEs and severity**

Package	Vulnerability ID	Severity	Installed Version	Fixed Version	Links
apk-tools	CVE-2021-36159	CRITICAL	2.12.5-r1	2.12.6-r0	https://github.com/ffnexus/android-sdk-extras-binaries/pull/1 https://lists.ubuntu.com/archives/ubuntu-security/2021-07/msg00001.html https://lists.ubuntu.com/archives/ubuntu-security/2021-07/msg00001.html Toggle more links
busybox	CVE-2021-42378	HIGH	1.33.1-r2	1.33.1-r6	https://access.redhat.com/security/cve/CVE-2021-42378 https://lists.ubuntu.com/archives/ubuntu-security/2021-07/msg00001.html https://lists.ubuntu.com/archives/ubuntu-security/2021-07/msg00001.html Toggle more links
busybox	CVE-2021-42379	HIGH	1.33.1-r2	1.33.1-r6	https://access.redhat.com/security/cve/CVE-2021-42379 https://lists.ubuntu.com/archives/ubuntu-security/2021-07/msg00001.html https://lists.ubuntu.com/archives/ubuntu-security/2021-07/msg00001.html Toggle more links
busybox	CVE-2021-42380	HIGH	1.33.1-r2	1.33.1-r6	https://access.redhat.com/security/cve/CVE-2021-42380 https://lists.ubuntu.com/archives/ubuntu-security/2021-07/msg00001.html https://lists.ubuntu.com/archives/ubuntu-security/2021-07/msg00001.html Toggle more links
busybox	CVE-2021-42381	HIGH	1.33.1-r2	1.33.1-r6	https://access.redhat.com/security/cve/CVE-2021-42381 https://lists.ubuntu.com/archives/ubuntu-security/2021-07/msg00001.html https://lists.ubuntu.com/archives/ubuntu-security/2021-07/msg00001.html Toggle more links
busybox	CVE-2021-42382	HIGH	1.33.1-r2	1.33.1-r6	https://access.redhat.com/security/cve/CVE-2021-42382 https://lists.ubuntu.com/archives/ubuntu-security/2021-07/msg00001.html https://lists.ubuntu.com/archives/ubuntu-security/2021-07/msg00001.html Toggle more links
busybox	CVE-2021-42383	HIGH	1.33.1-r2	1.33.1-r6	https://access.redhat.com/security/cve/CVE-2021-42383 https://lists.ubuntu.com/archives/ubuntu-security/2021-07/msg00001.html https://lists.ubuntu.com/archives/ubuntu-security/2021-07/msg00001.html Toggle more links
busybox	CVE-2021-42384	HIGH	1.33.1-r2	1.33.1-r6	https://access.redhat.com/security/cve/CVE-2021-42384 https://lists.ubuntu.com/archives/ubuntu-security/2021-07/msg00001.html https://lists.ubuntu.com/archives/ubuntu-security/2021-07/msg00001.html Toggle more links
busybox	CVE-2021-42385	HIGH	1.33.1-r2	1.33.1-r6	https://access.redhat.com/security/cve/CVE-2021-42385 https://lists.ubuntu.com/archives/ubuntu-security/2021-07/msg00001.html https://lists.ubuntu.com/archives/ubuntu-security/2021-07/msg00001.html Toggle more links

- **GitHub PR showing image tag update in deployment.yaml**



- **SonarQube project analysis for ArgoCD deployment**



- **Kubernetes pods monitoring with Prometheus targets**

Prometheus Alerts Graph Status Help

Targets

All scrape pools Unhealthy Collapse All Filter by endpoint or labels

Unknown Unhealthy Healthy

serviceMonitor/prometheus/stable-kube-prometheus-sta-alertmanager/0 (1/1 up)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://172.31.95.15:9093/metrics	UP	container="alertmanager" endpoint="http-web" instance="172.31.95.15:9093" job="stable-kube-prometheus-sta-alertmanager" namespace="prometheus" pod="alertmanager-stable-kube-prometheus-sta-alertmanager-0" service="stable-kube-prometheus-sta-alertmanager"	3m 20s ago	4.665ms	

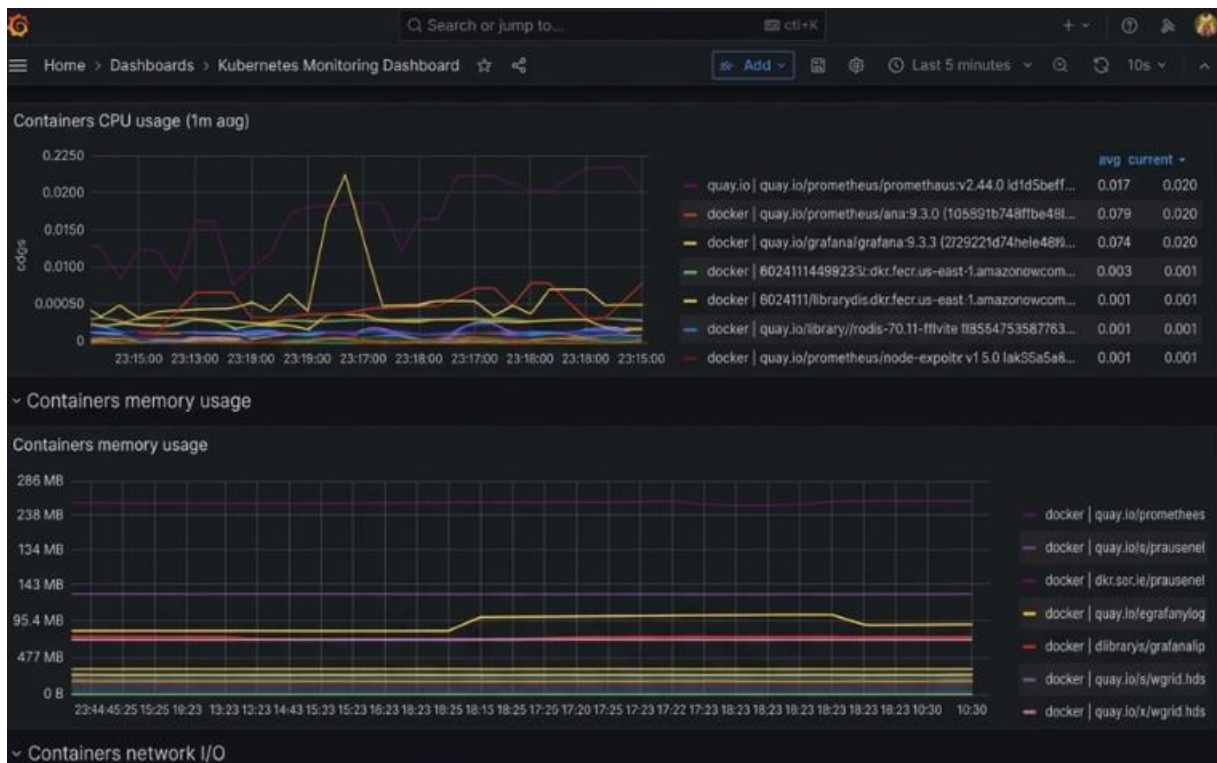
serviceMonitor/prometheus/stable-kube-prometheus-sta-apiserver/0 (2/2 up)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
https://172.31.31.102/metrics	UP	endpoint="https" instance="172.31.31.102:443" job="apiserver" namespace="default" service="kubernetes"	3m 39s ago	92.801ms	
https://172.31.88.131/metrics	UP	endpoint="https" instance="172.31.88.131:443" job="apiserver" namespace="default" service="kubernetes"	3m 21s ago	123.674ms	

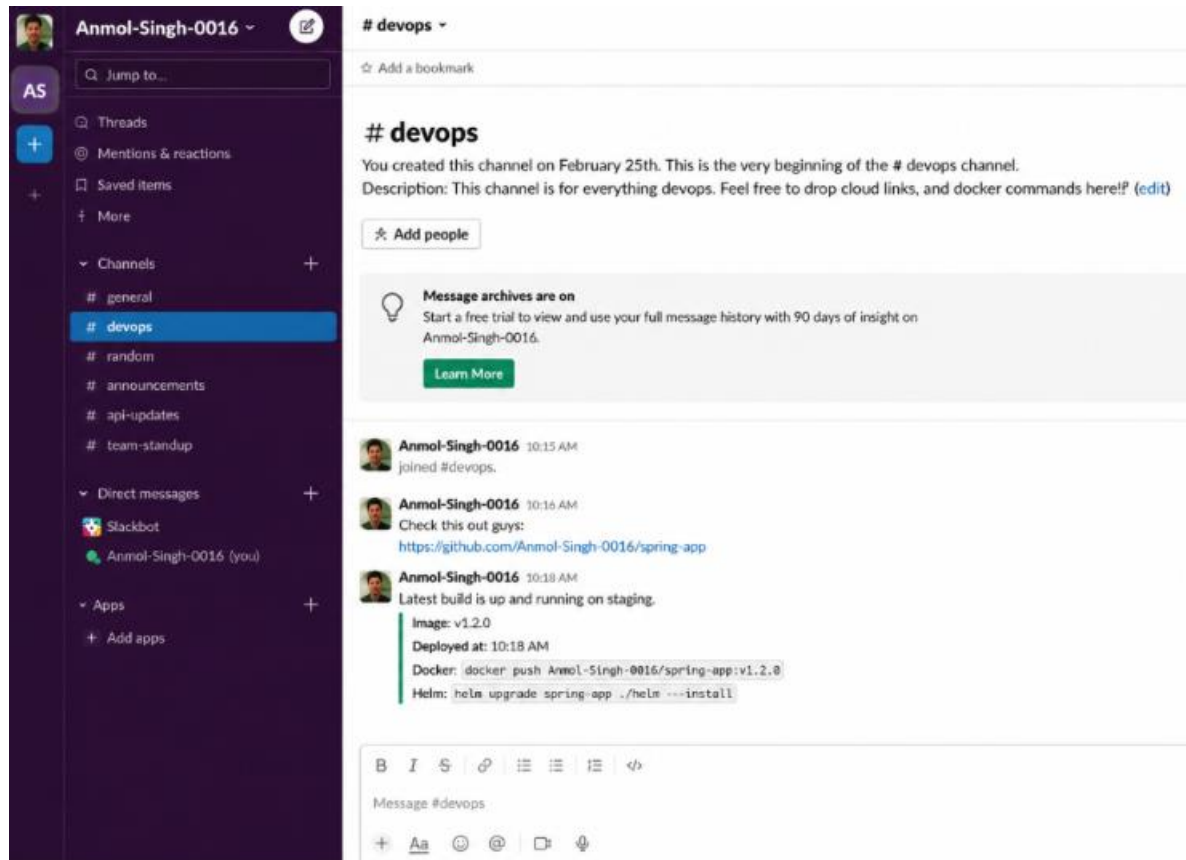
serviceMonitor/prometheus/stable-kube-prometheus-sta-coredns/0 (2/2 up)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://172.31.88.175:9153/metrics	UP	container="coredns" endpoint="http-metrics" instance="172.31.88.175:9153" job="coredns" namespace="kube-system" pod="coredns-7899945769-4hcz7" service="stable-kube-prometheus-sta-coredns"	3m 38s ago	3.030ms	
http://172.31.89.191:9153/metrics	UP	container="coredns" endpoint="http-metrics" instance="172.31.89.191:9153" job="coredns"	3m 32s ago	2.829ms	

- **Grafana dashboard showing container CPU usage**



- Slack channel #devops with build notifications



16. Conclusion

- This project successfully demonstrates the implementation of a complete **Production-Grade DevSecOps CI/CD Pipeline** using modern cloud-native technologies.
- By integrating Jenkins, SonarQube, Docker, Trivy, ArgoCD, Kubernetes, Prometheus, Grafana, Slack, JFrog Artifactory, and Hashicorp Vault, the system achieves:
Automation ,Scalability ,Security ,Continuous Integration / Continuous Deployment ,Infrastructure Observability .
- The project showcases how DevOps and GitOps principles improve software delivery speed, deployment reliability, and operational efficiency in modern enterprise environments.